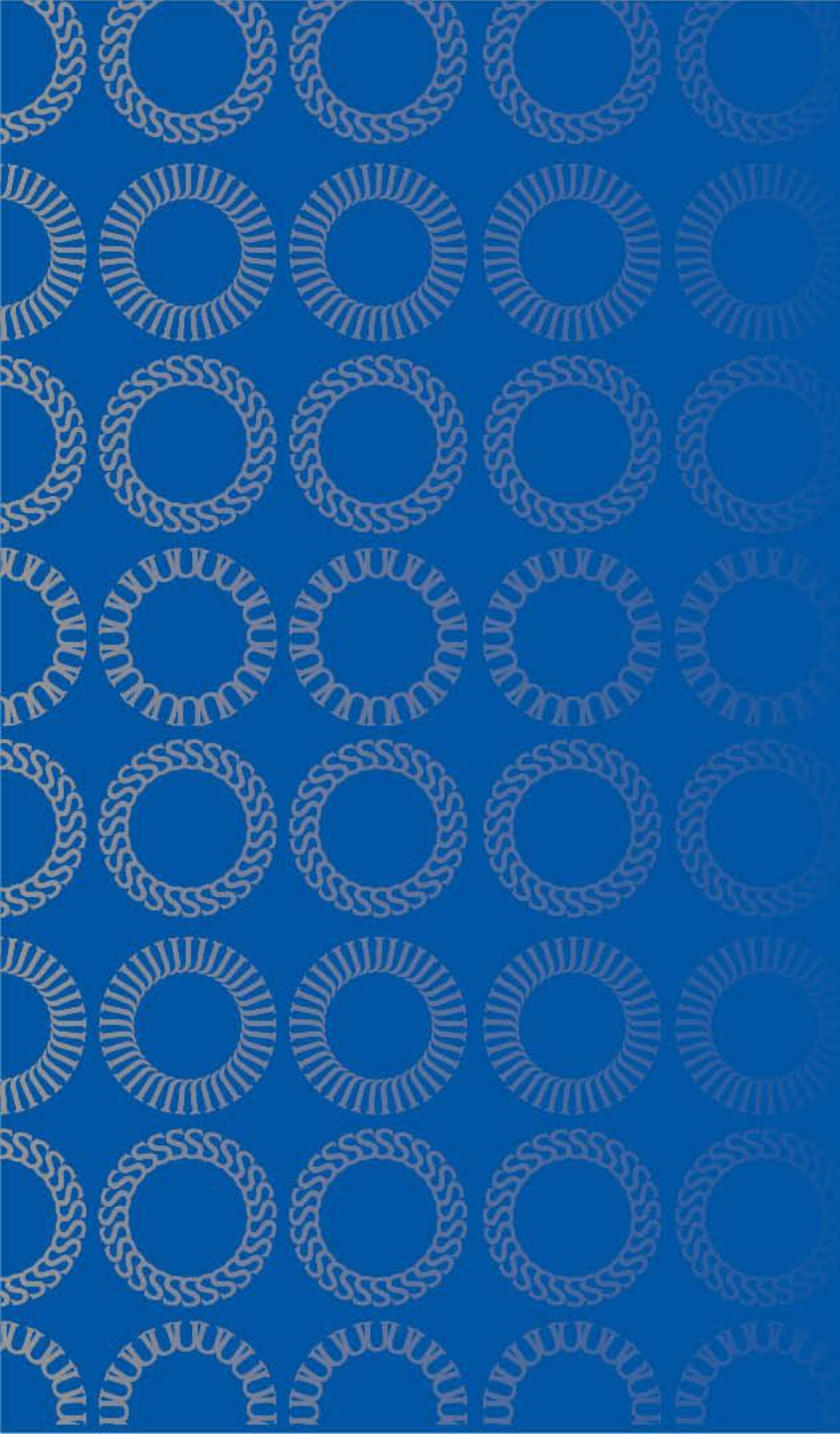




# DATA 255 Deep Learning Technologies

Oct 2, 2025

**Taehee Jeong, Ph.D.**

# Topics(1)

- CNN architectures
  - LeNet-5
  - AlexNet
  - VGG
  - Googlenet/Inceptionv3
  - ResNet
  - Mobilenet
- Data Augmentation
- Regularization for overfitting

# Main applications in CNN

- Image classification
- Object detection
- Image segmentation

Gradient-based learning applied to document recognition, LeCun et al., 1998.

# LeNet-5

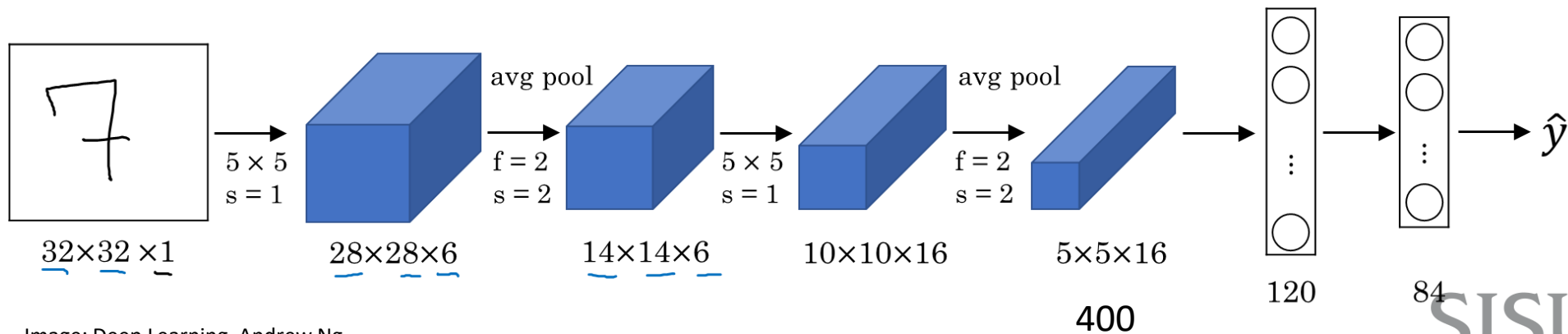
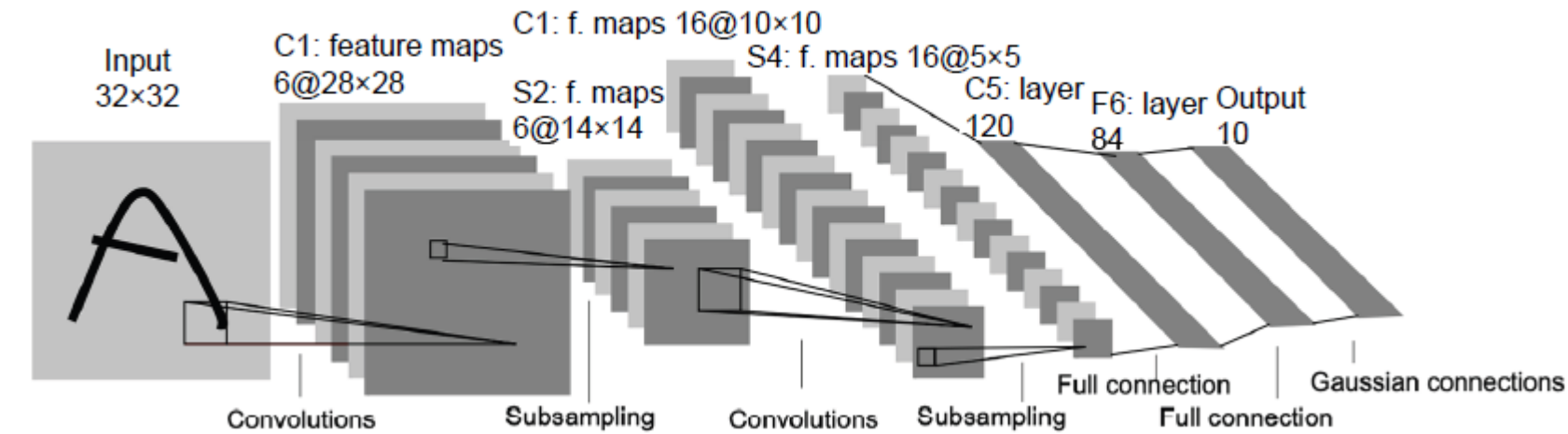


Image: Deep Learning, Andrew Ng

Gradient-based learning applied to document recognition, LeCun et al., 1998.

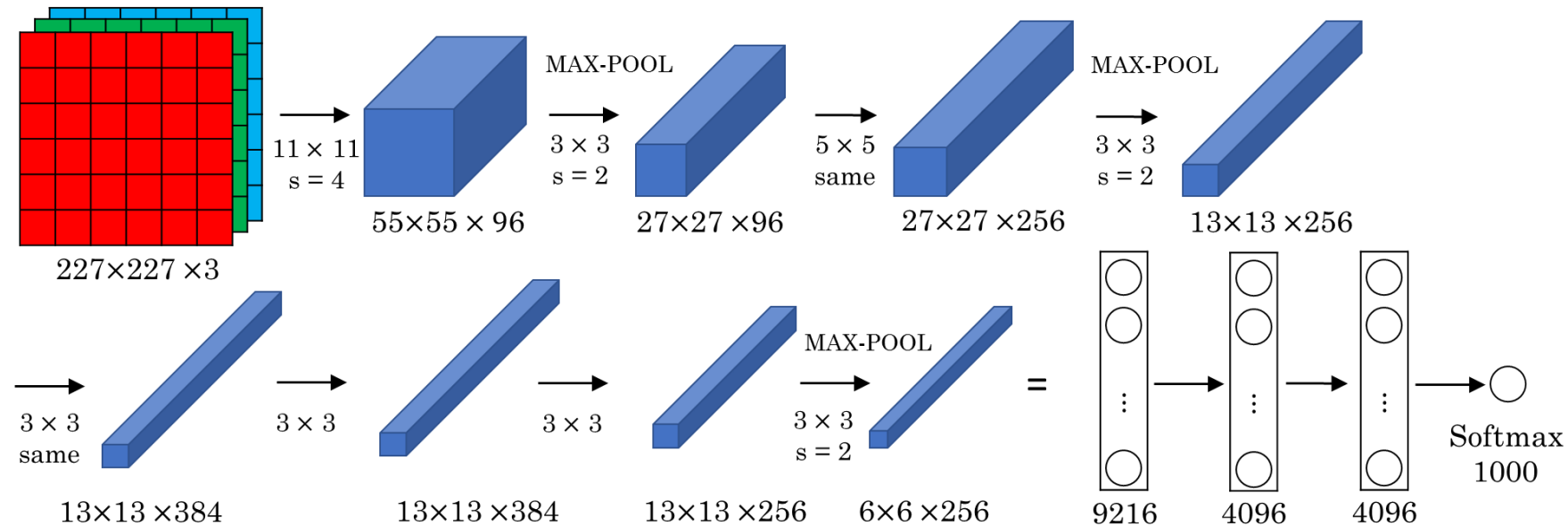
# LeNet-5

created by Yann LeCun in 1998 and widely used for handwritten digit recognition (MNIST)

| Layer           | feature map | kernel | channel | stride | activation |
|-----------------|-------------|--------|---------|--------|------------|
| Input           | 32x32x1     |        |         |        |            |
| Convolution     | 28x28x6     | 5x5    | 6       | 1      | tanh       |
| Avg Pooling     | 14x14x6     | 2x2    |         | 2      |            |
| Convolution     | 10x10x16    | 5x5    | 16      | 1      | tanh       |
| Avg Pooling     | 5x5x16      | 2x2    |         | 2      |            |
| Flatten         | 400         |        |         |        |            |
| Fully Connected | 120         |        |         |        | tanh       |
| Fully Connected | 84          |        |         |        | tanh       |
| Fully Connected | 10          |        |         |        | RBF        |



# AlexNet



ImageNet Classification with Deep Convolutional Neural Networks, A. Krizhevsky et al. (2012)

# AlexNet

won the 2012 ImageNet ILSVRC challenge with 83% accuracy.

| Layer           | feature map | kernel | channel | stride | Padding | activation |
|-----------------|-------------|--------|---------|--------|---------|------------|
| Input           | 224x224x3   |        |         |        |         |            |
| Convolution     | 55x55x96    | 11x11  | 96      | 4      | same    | ReLU       |
| Max Pooling     | 27x27x96    | 3x3    |         | 2      | valid   |            |
| Convolution     | 27x27x256   | 5x5    | 256     | 1      | same    | ReLU       |
| Max Pooling     | 13x13x256   | 3x3    |         | 2      | valid   |            |
| Convolution     | 13x13x384   | 3x3    | 384     | 1      | same    | ReLU       |
| Convolution     | 13x13x384   | 3x3    | 384     | 1      | same    | ReLU       |
| Convolution     | 13x13x256   | 3x3    | 256     | 1      | same    | ReLU       |
| Max Pooling     | 6x6x256     | 3x3    |         | 2      | valid   |            |
| Flatten         | 9216        |        |         |        |         |            |
| Fully Connected | 4096        |        |         |        |         | ReLU       |
| Fully Connected | 4096        |        |         |        |         | ReLU       |
| Fully Connected | 1000        |        |         |        |         | Softmax    |

Very deep convolutional networks for large-scale image recognition, Simonyan & Zisserman 2015.

# VGG-16

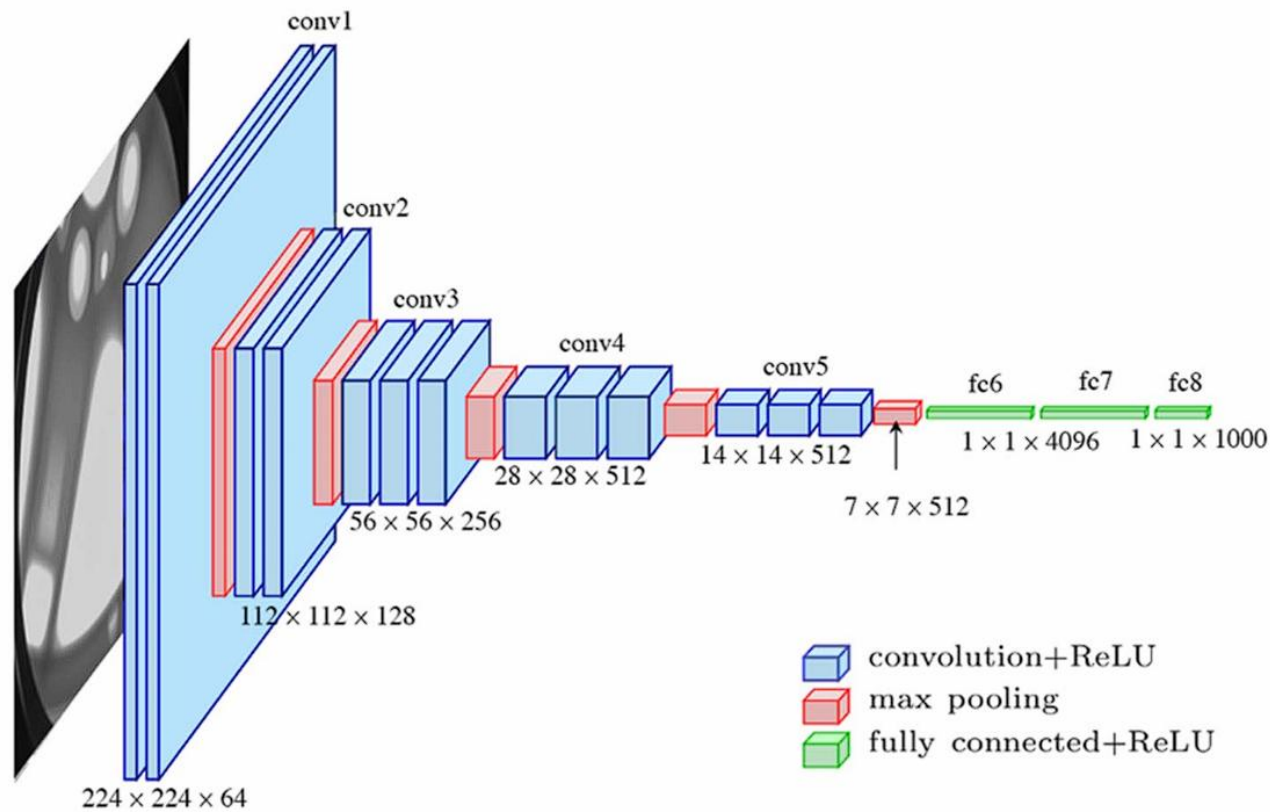


Image: <https://viso.ai/deep-learning/googlenet-explained-the-inception-model-that-won-imagenet>

Very deep convolutional networks for large-scale image recognition, Simonyan & Zisserman 2015.

# VGG-16

CONV =  $3 \times 3$  filter,  $s = 1$ , same convolution

MAX-POOL =  $2 \times 2$ ,  $s = 2$

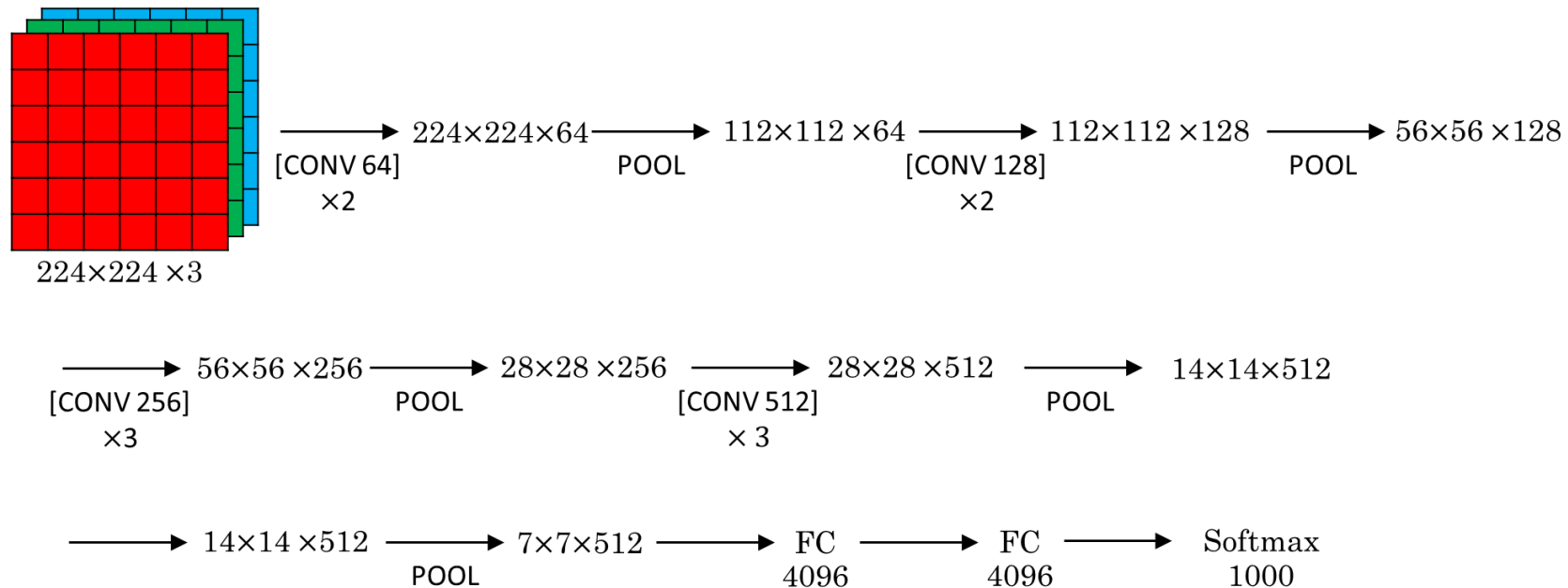


Image: Deep Learning, Andrew Ng

Flatten layer:

$[7, 7, 512] \rightarrow 25088 \rightarrow [4096]$

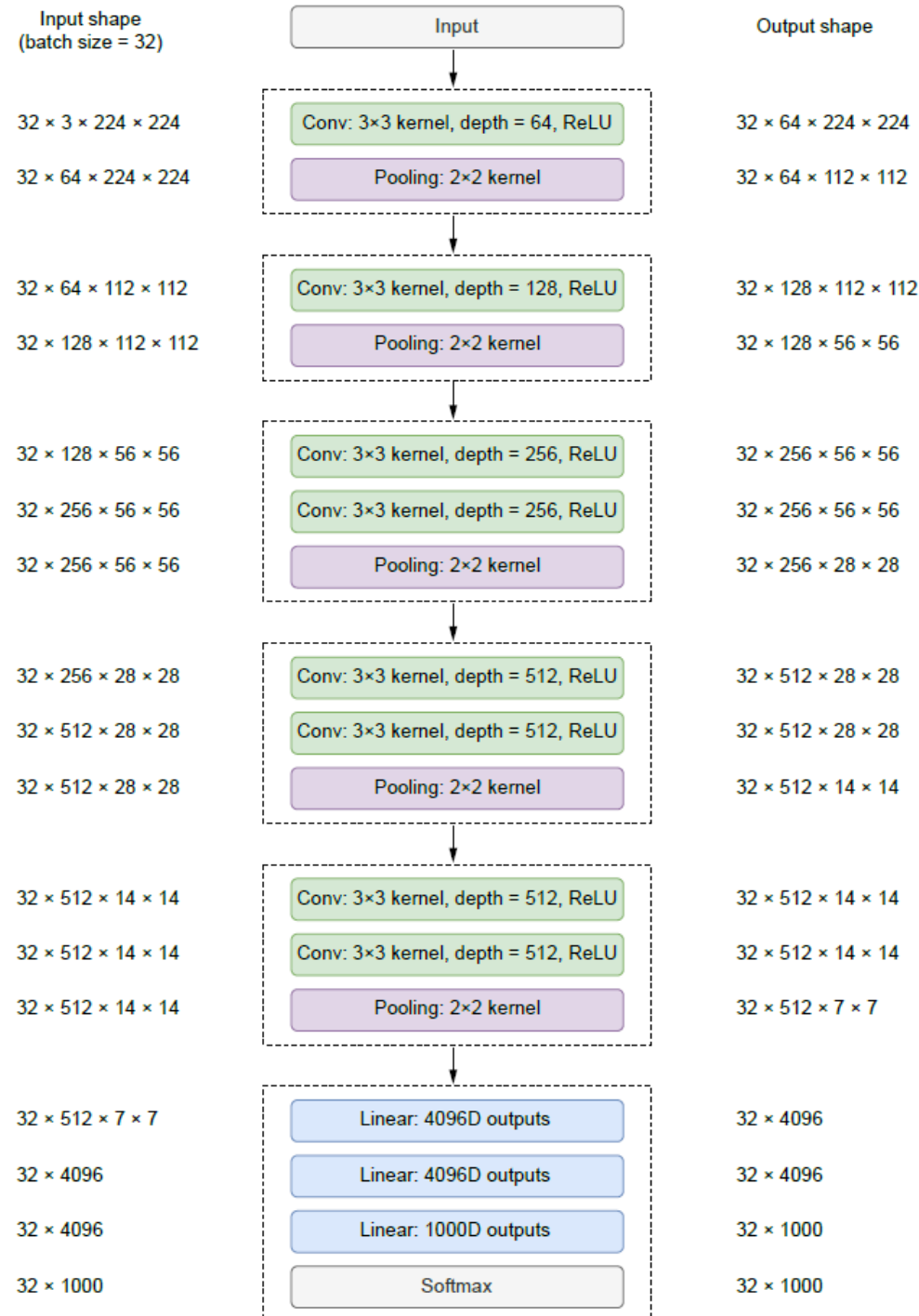
Very deep convolutional networks for large-scale image recognition, Simonyan & Zisserman 2015.

# VGG

| ConvNet Configuration               |                        |                               |                                            |                                            |                                                         |
|-------------------------------------|------------------------|-------------------------------|--------------------------------------------|--------------------------------------------|---------------------------------------------------------|
| A                                   | A-LRN                  | B                             | C                                          | D                                          | E                                                       |
| 11 weight layers                    | 11 weight layers       | 13 weight layers              | 16 weight layers                           | 16 weight layers                           | 19 weight layers                                        |
| input ( $224 \times 224$ RGB image) |                        |                               |                                            |                                            |                                                         |
| conv3-64                            | conv3-64<br><b>LRN</b> | conv3-64<br><b>conv3-64</b>   | conv3-64<br>conv3-64                       | conv3-64<br>conv3-64                       | conv3-64<br>conv3-64                                    |
| maxpool                             |                        |                               |                                            |                                            |                                                         |
| conv3-128                           | conv3-128              | conv3-128<br><b>conv3-128</b> | conv3-128<br>conv3-128                     | conv3-128<br>conv3-128                     | conv3-128<br>conv3-128                                  |
| maxpool                             |                        |                               |                                            |                                            |                                                         |
| conv3-256<br>conv3-256              | conv3-256<br>conv3-256 | conv3-256<br>conv3-256        | conv3-256<br>conv3-256<br><b>conv1-256</b> | conv3-256<br>conv3-256<br><b>conv3-256</b> | conv3-256<br>conv3-256<br>conv3-256<br><b>conv3-256</b> |
| maxpool                             |                        |                               |                                            |                                            |                                                         |
| conv3-512<br>conv3-512              | conv3-512<br>conv3-512 | conv3-512<br>conv3-512        | conv3-512<br>conv3-512<br><b>conv1-512</b> | conv3-512<br>conv3-512<br><b>conv3-512</b> | conv3-512<br>conv3-512<br>conv3-512<br><b>conv3-512</b> |
| maxpool                             |                        |                               |                                            |                                            |                                                         |
| conv3-512<br>conv3-512              | conv3-512<br>conv3-512 | conv3-512<br>conv3-512        | conv3-512<br>conv3-512<br><b>conv1-512</b> | conv3-512<br>conv3-512<br><b>conv3-512</b> | conv3-512<br>conv3-512<br>conv3-512<br><b>conv3-512</b> |
| maxpool                             |                        |                               |                                            |                                            |                                                         |
| FC-4096                             |                        |                               |                                            |                                            |                                                         |
| FC-4096                             |                        |                               |                                            |                                            |                                                         |
| FC-1000                             |                        |                               |                                            |                                            |                                                         |
| soft-max                            |                        |                               |                                            |                                            |                                                         |

Activation function: ReLU

# VGG-11



Source: Math and Architectures of Deep Learning, Krishnedu Chauhury

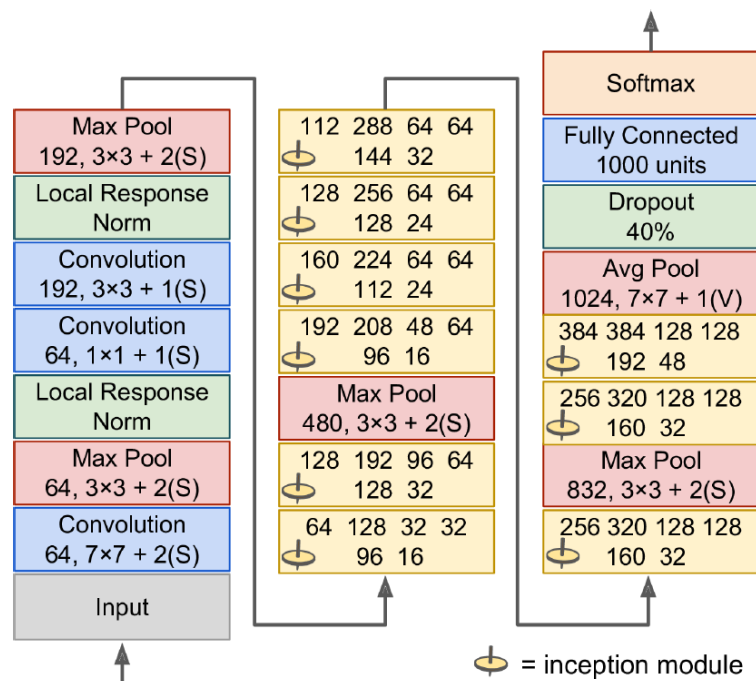
Very deep convolutional networks for large-scale image recognition, Simonyan & Zisserman 2015.

# VGG-11

| Layer           | feature map | kernel | channel | stride | Padding | activation |
|-----------------|-------------|--------|---------|--------|---------|------------|
| Input           | 224x224x3   |        |         |        |         |            |
| Convolution     | 224x224x64  | 3x3    | 64      | 1      | same    | ReLU       |
| Max Pooling     | 112x112x64  | 2x2    |         | 2      | valid   |            |
| Convolution     | 112x112x128 | 3x3    | 128     | 1      | same    | ReLU       |
| Max Pooling     | 56x56x128   | 2x2    |         | 2      | valid   |            |
| Convolution     | 56x56x256   | 3x3    | 256     | 1      | same    | ReLU       |
| Convolution     | 56x56x256   | 3x3    | 256     | 1      | same    | ReLU       |
| Max Pooling     | 28x28x256   | 2x2    |         | 2      | valid   |            |
| Convolution     | 28x28x512   | 3x3    | 512     | 1      | same    | ReLU       |
| Convolution     | 28x28x512   | 3x3    | 512     | 1      | same    | ReLU       |
| Max Pooling     | 28x28x256   | 2x2    |         | 2      | valid   |            |
| Convolution     | 14x14x512   | 3x3    | 512     | 1      | same    | ReLU       |
| Convolution     | 14x14x512   | 3x3    | 512     | 1      | same    | ReLU       |
| Max Pooling     | 7x7x512     | 2x2    |         | 2      | valid   |            |
| Flatten         | 25088       |        |         |        |         |            |
| Fully Connected | 4096        |        |         |        |         | ReLU       |
| Fully Connected | 4096        |        |         |        |         | ReLU       |
| Fully Connected | 1000        |        |         |        |         | Softmax    |

# GoogLeNet(inceptionv1)

won the ILSVRC 2014 challenge with 93% accuracy.



| type           | patch size/<br>stride | output<br>size             |
|----------------|-----------------------|----------------------------|
| convolution    | $7 \times 7 / 2$      | $112 \times 112 \times 64$ |
| max pool       | $3 \times 3 / 2$      | $56 \times 56 \times 64$   |
| convolution    | $3 \times 3 / 1$      | $56 \times 56 \times 192$  |
| max pool       | $3 \times 3 / 2$      | $28 \times 28 \times 192$  |
| inception (3a) |                       | $28 \times 28 \times 256$  |
| inception (3b) |                       | $28 \times 28 \times 480$  |
| max pool       | $3 \times 3 / 2$      | $14 \times 14 \times 480$  |
| inception (4a) |                       | $14 \times 14 \times 512$  |
| inception (4b) |                       | $14 \times 14 \times 512$  |
| inception (4c) |                       | $14 \times 14 \times 512$  |
| inception (4d) |                       | $14 \times 14 \times 528$  |
| inception (4e) |                       | $14 \times 14 \times 832$  |
| max pool       | $3 \times 3 / 2$      | $7 \times 7 \times 832$    |
| inception (5a) |                       | $7 \times 7 \times 832$    |
| inception (5b) |                       | $7 \times 7 \times 1024$   |
| avg pool       | $7 \times 7 / 1$      | $1 \times 1 \times 1024$   |
| dropout (40%)  |                       | $1 \times 1 \times 1024$   |
| linear         |                       | $1 \times 1 \times 1000$   |
| softmax        |                       | $1 \times 1 \times 1000$   |

# GoogLeNet(inceptionv1)

- Key features:
  - 1×1 Convolution
  - Global Average Pooling
  - Inception Module

# 1x 1 convolution

In the case of one channel

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 2 | 3 | 6 | 5 | 8 |
| 3 | 5 | 5 | 1 | 3 | 4 |
| 2 | 1 | 3 | 4 | 9 | 3 |
| 4 | 7 | 8 | 5 | 7 | 9 |
| 1 | 5 | 3 | 7 | 4 | 8 |
| 5 | 4 | 9 | 8 | 3 | 5 |

6 × 6

\*

|   |
|---|
| 2 |
|---|

1 × 1

=

|  |  |  |  |  |  |
|--|--|--|--|--|--|
|  |  |  |  |  |  |
|  |  |  |  |  |  |
|  |  |  |  |  |  |
|  |  |  |  |  |  |
|  |  |  |  |  |  |
|  |  |  |  |  |  |

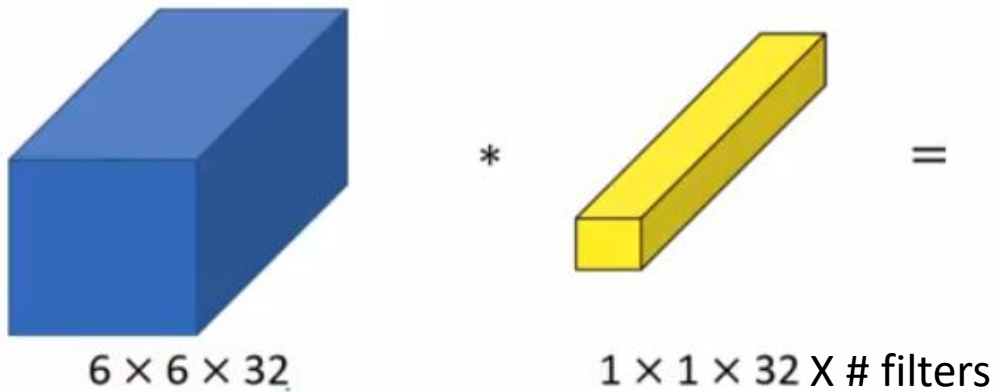
6 × 6

Image: Deep Learning, Andrew Ng

Source: Network in network, Lin et al., 2013.

# 1x 1 convolution

In the case of multiple channels



Convolution : sum of individual pixel over all channels

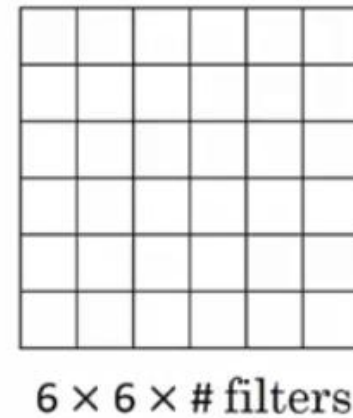


Image: Deep Learning, Andrew Ng

**Convolution  
+ Activation**

Source: Network in network, Lin et al., 2013.

# 1x 1 convolution

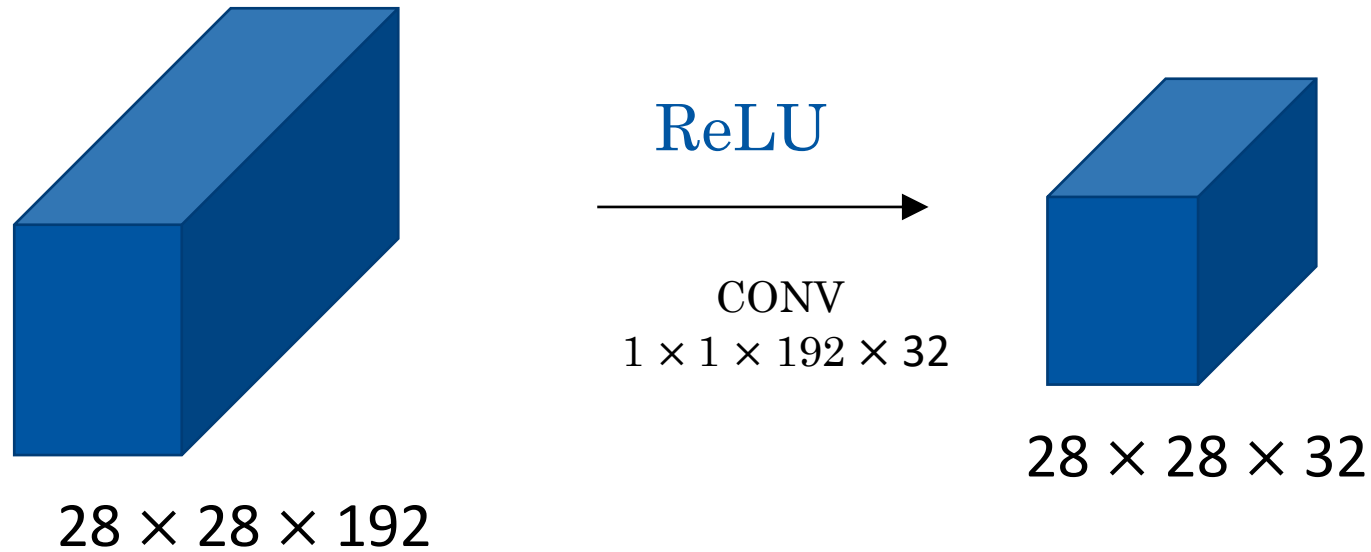


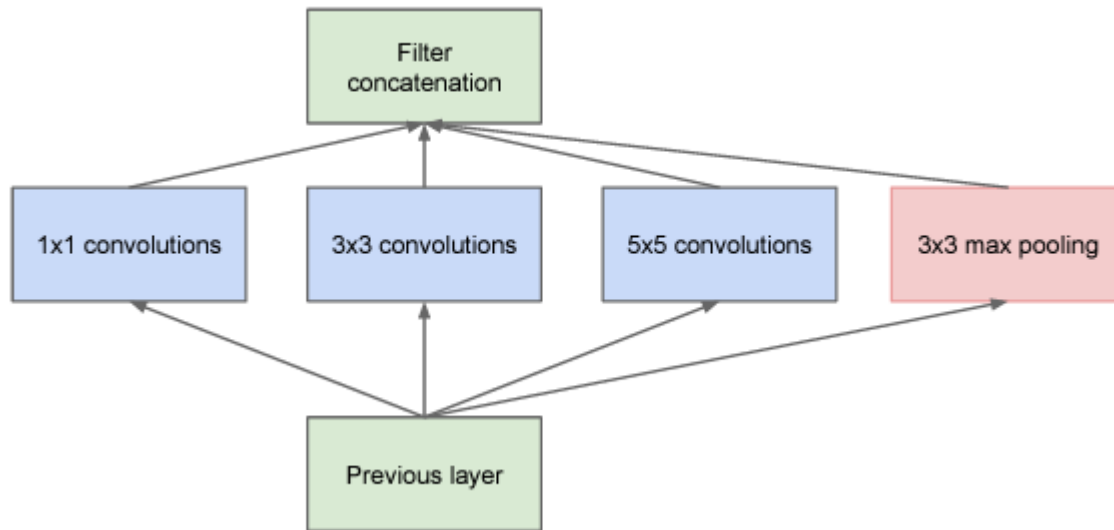
Image: Deep Learning, Andrew Ng

# Global average pooling

It performs an average operation across the Width and Height of each filter channel separately. This reduces the feature map to a vector that is equal to the size of the number of channels. It is computationally efficient compared to flatten layer.

For example,  
a feature map: [10,10, 32] (Width, Height, Channels).  
→ Summarize each channel across the entire feature map  
→ The output : [1,1,32]

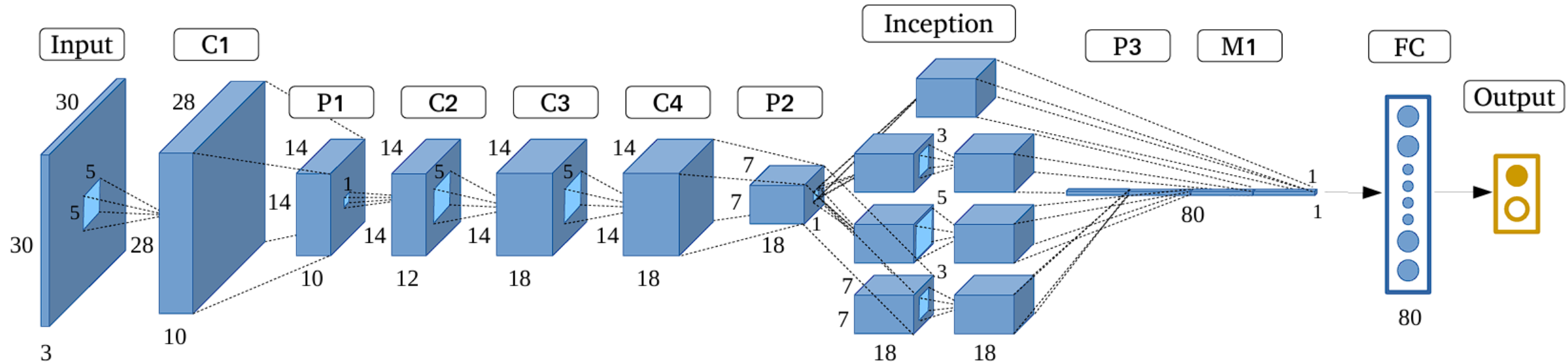
# Inception Module



Inception module, naive version

- Inception module:
  - parallel convolutions with various filter sizes ( $1\times 1$ ,  $3\times 3$ , and  $5\times 5$ )
  - The outputs from these filters are then concatenated
  - Multi-Level Feature Extraction:  
It captures a wide range of features in the input data

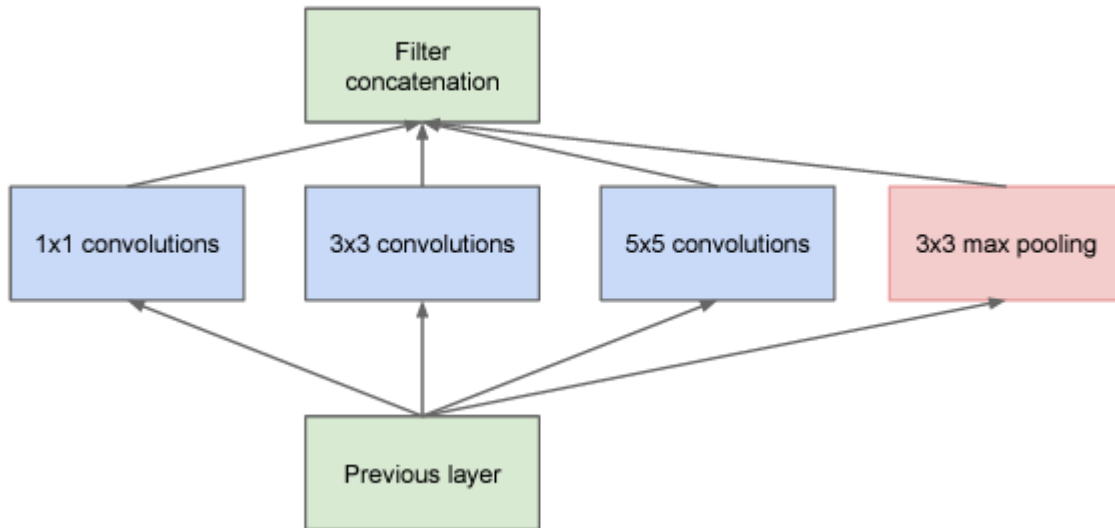
# GoogLeNet(inceptionv1)



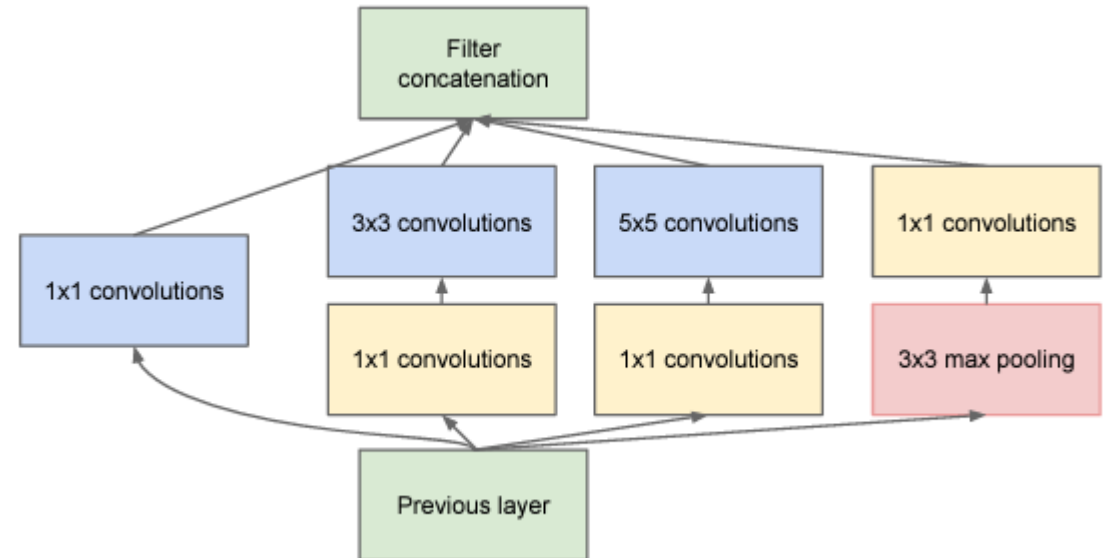
# Inceptionv3

| type                 | patch size/stride<br>or remarks | input size                 |
|----------------------|---------------------------------|----------------------------|
| conv                 | $3 \times 3 / 2$                | $299 \times 299 \times 3$  |
| conv                 | $3 \times 3 / 1$                | $149 \times 149 \times 32$ |
| conv padded          | $3 \times 3 / 1$                | $147 \times 147 \times 32$ |
| pool                 | $3 \times 3 / 2$                | $147 \times 147 \times 64$ |
| conv                 | $3 \times 3 / 1$                | $73 \times 73 \times 64$   |
| conv                 | $3 \times 3 / 2$                | $71 \times 71 \times 80$   |
| conv                 | $3 \times 3 / 1$                | $35 \times 35 \times 192$  |
| $3 \times$ Inception | As in figure 5                  | $35 \times 35 \times 288$  |
| $5 \times$ Inception | As in figure 6                  | $17 \times 17 \times 768$  |
| $2 \times$ Inception | As in figure 7                  | $8 \times 8 \times 1280$   |
| pool                 | $8 \times 8$                    | $8 \times 8 \times 2048$   |
| linear               | logits                          | $1 \times 1 \times 2048$   |
| softmax              | classifier                      | $1 \times 1 \times 1000$   |

# Efficient Grid Size Reduction



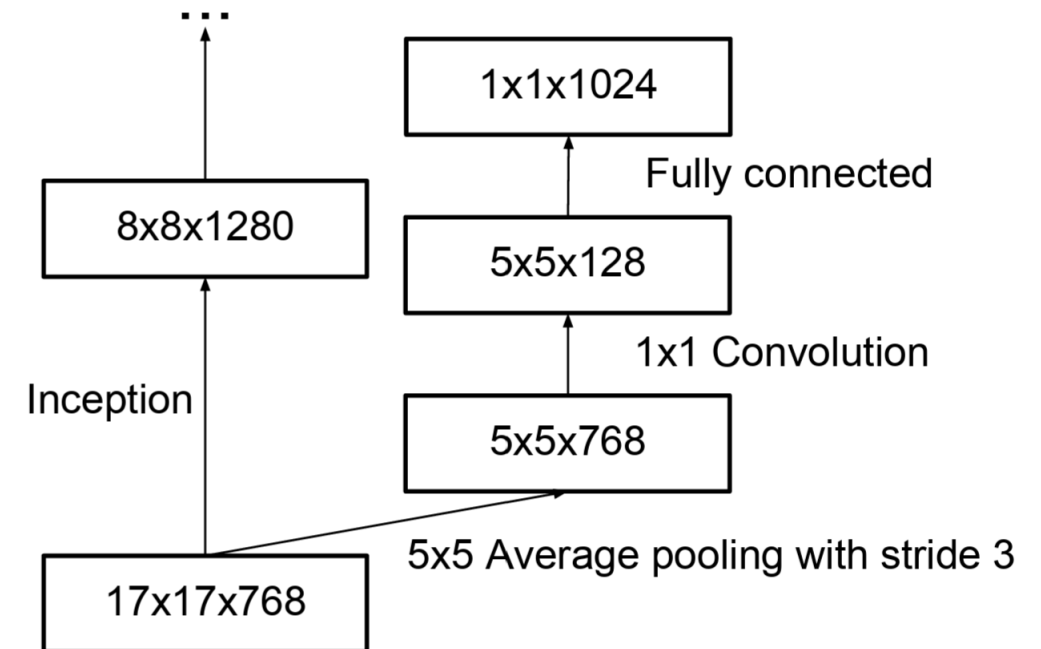
Inception module, naive version



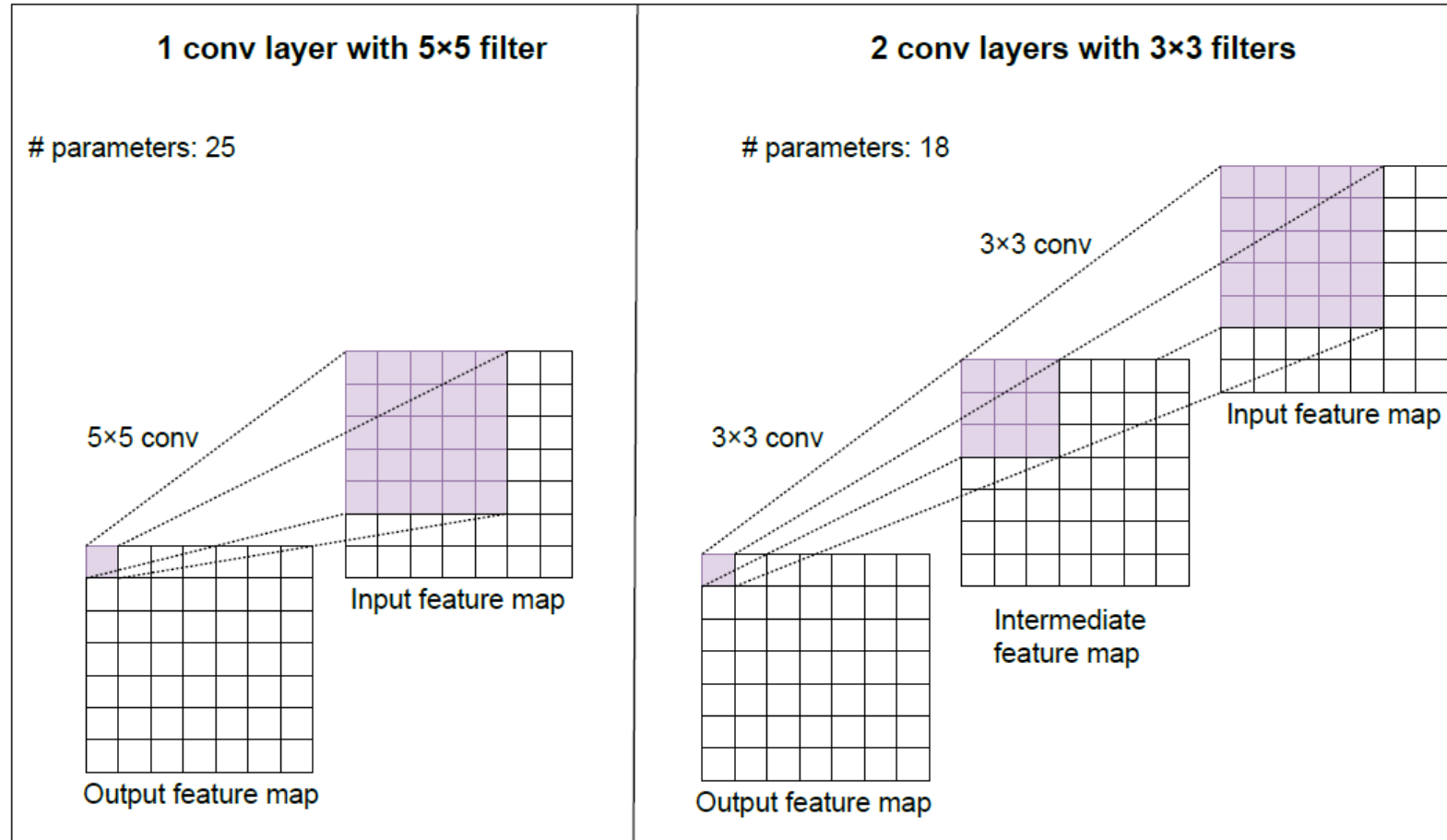
Inception module with dimension reductions

# Efficient Grid Size Reduction

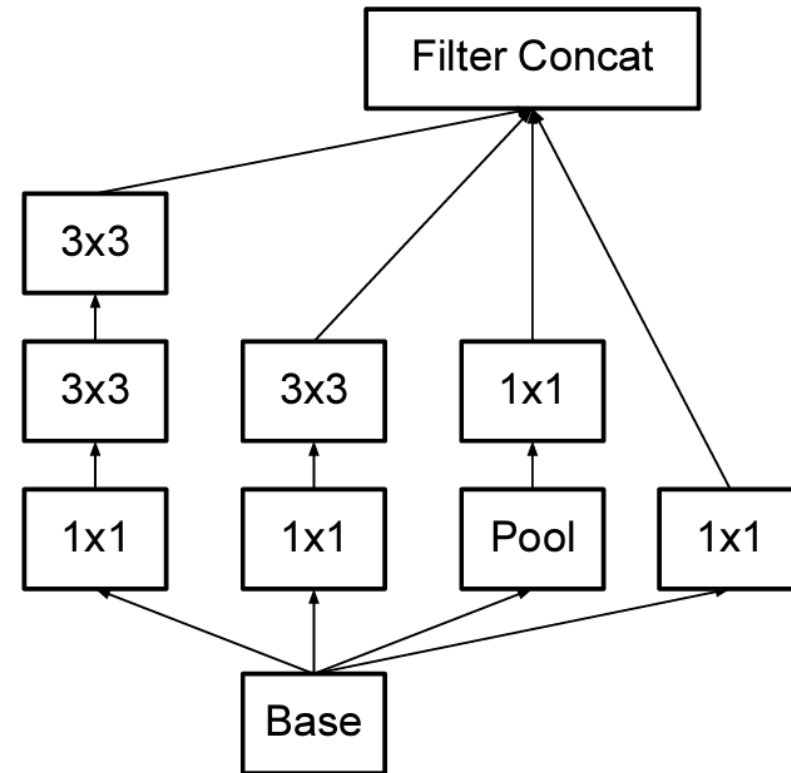
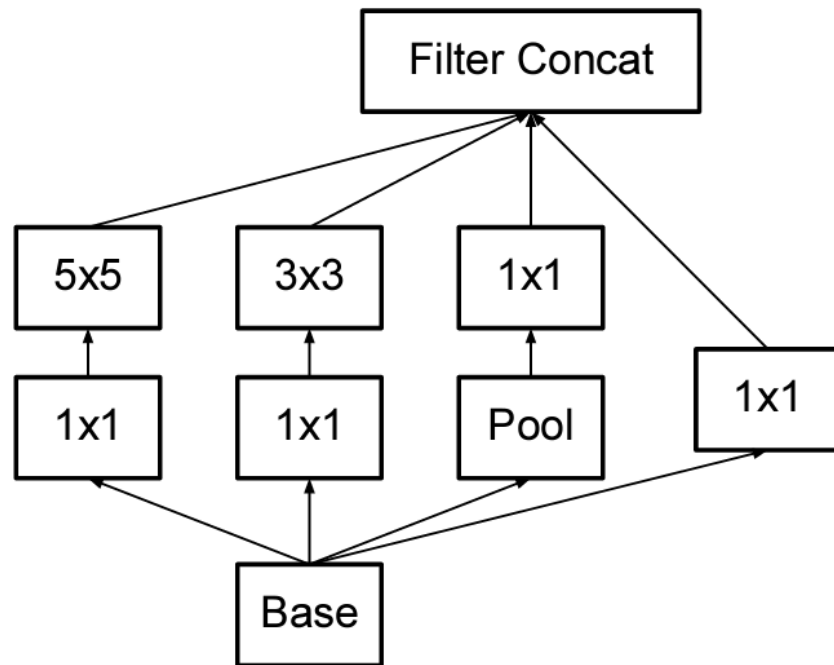
By using  $1 \times 1$  convolutions, the module reduces dimensionality before applying the more expensive  $3 \times 3$  and  $5 \times 5$  convolutions and pooling operations.



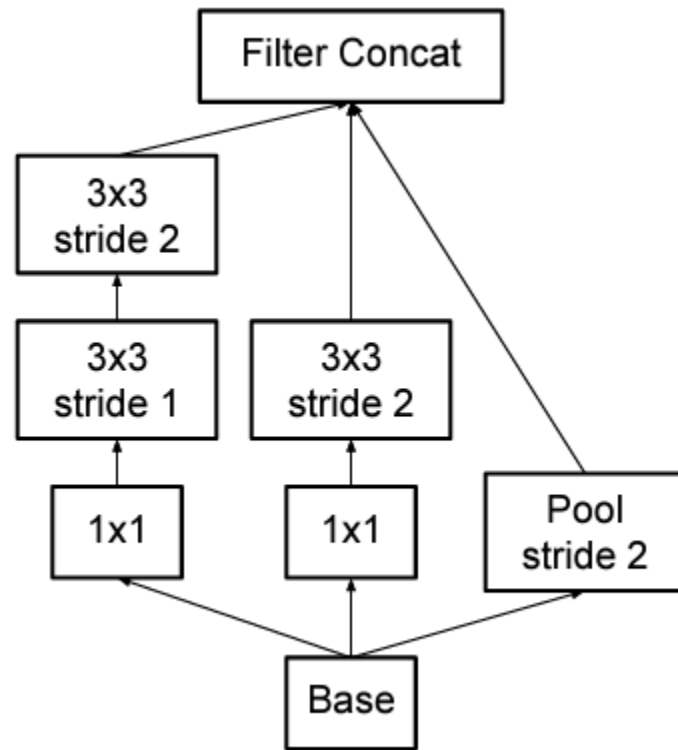
# Factorization into smaller convolutions



# Factorization into smaller convolutions



# Inceptionv3

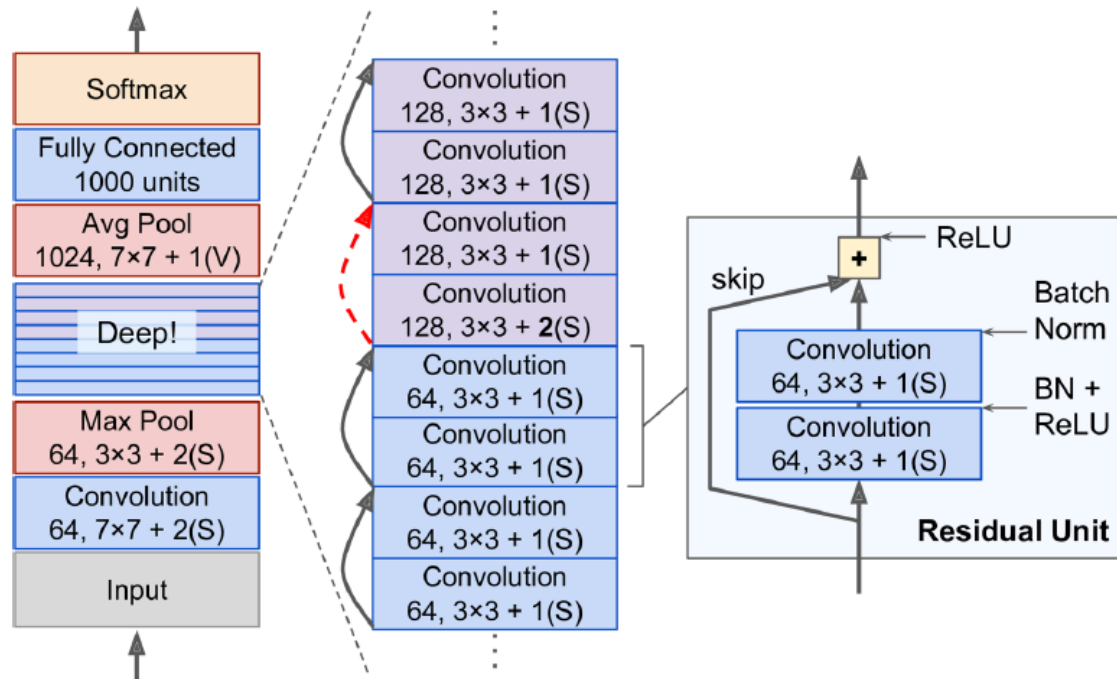


- Factorization into smaller convolutions
- Efficient Grid Size Reduction

Deep residual networks for image recognition, He et al., 2015.

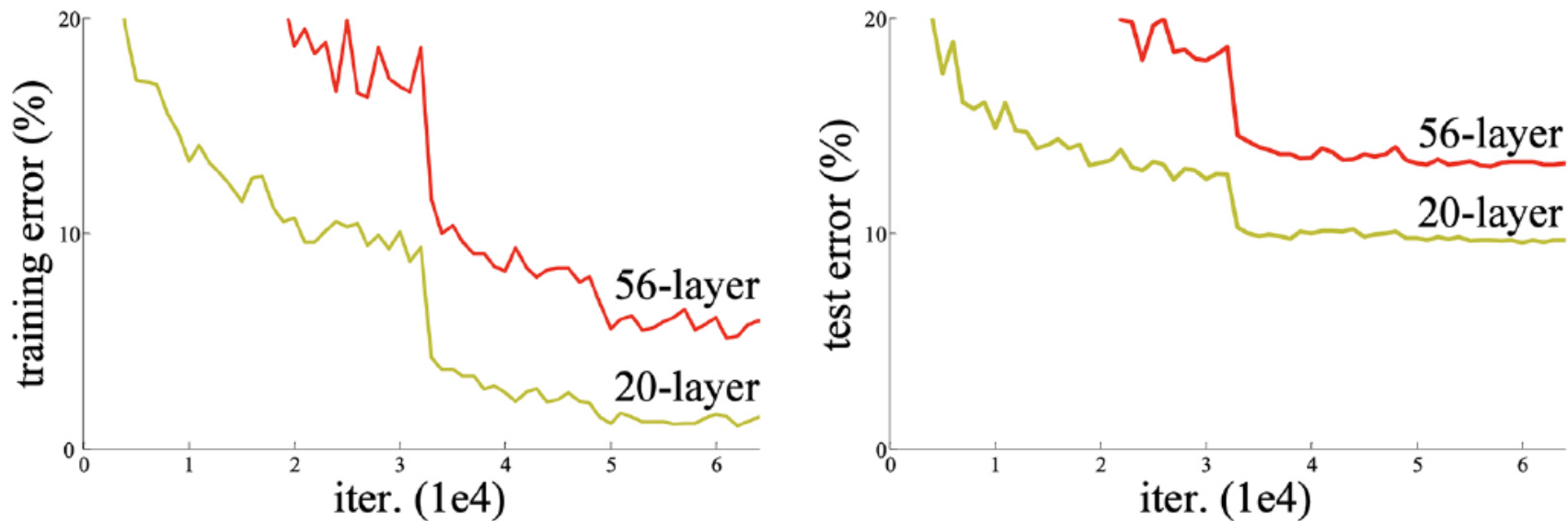
# ResNet (residual network)

won the ILSVRC 2015 challenge with 96.4% accuracy.



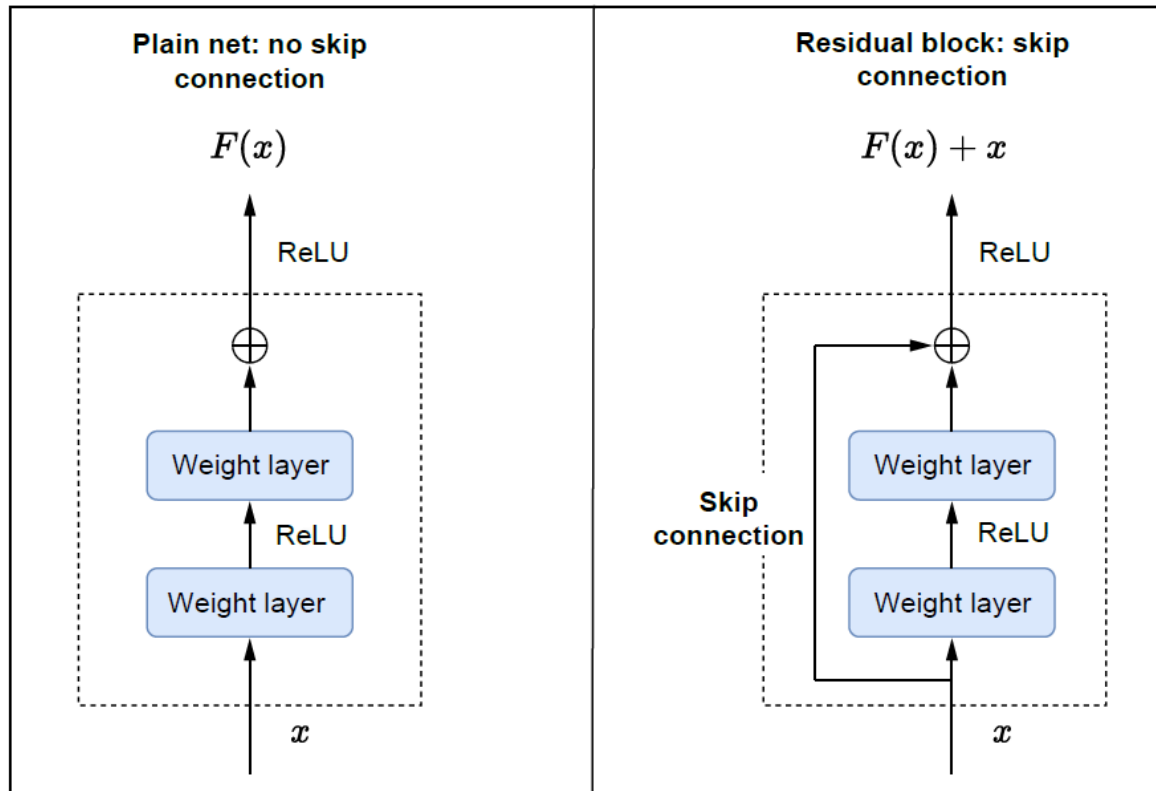
Source: *Hands-On Machine Learning with Scikit-Learn & TensorFlow*

# Why stacking layers to add depth does not scale



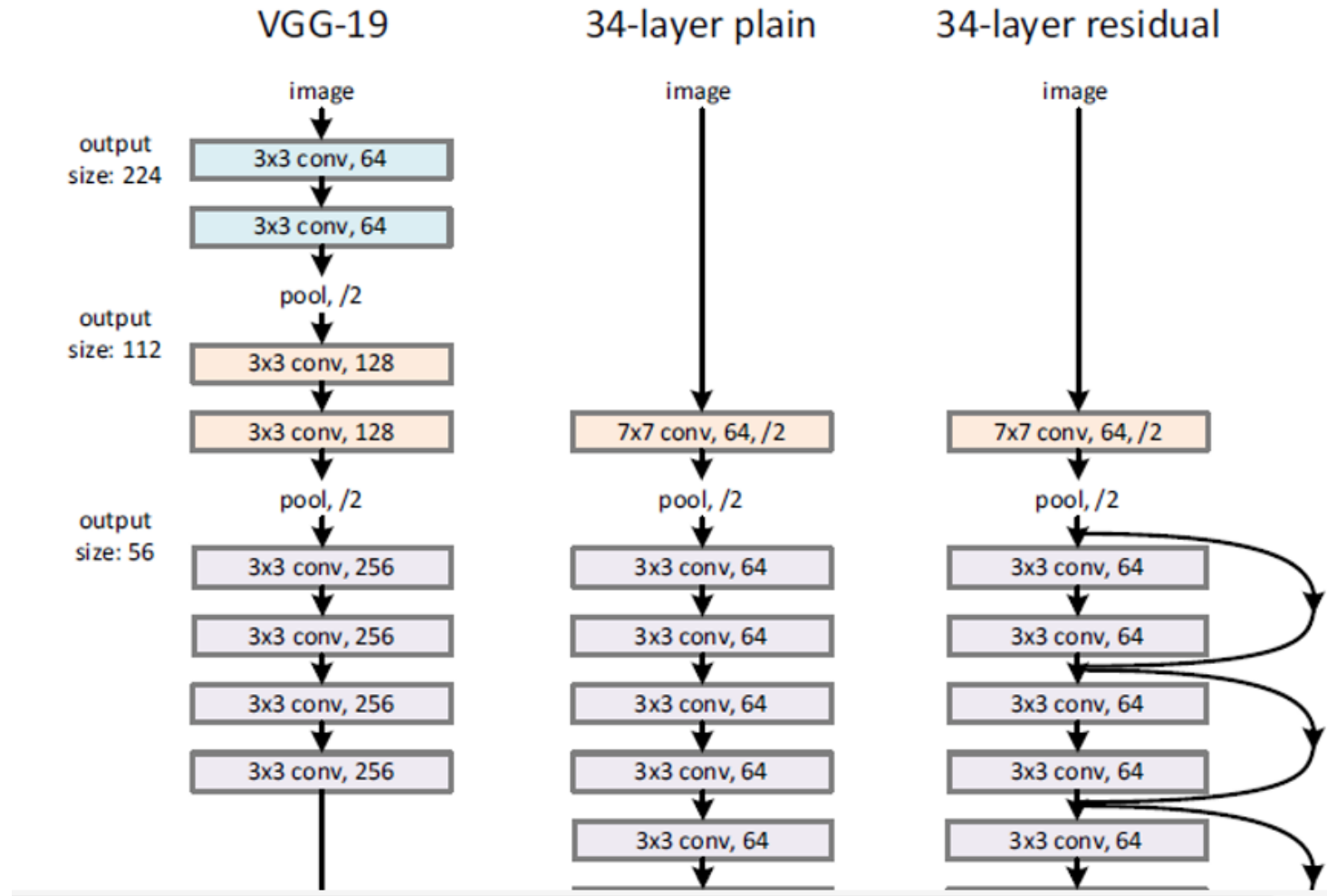
- we expect deeper networks to have more expressive power and hence higher accuracies/lower error rates than their shallower counterparts
- with the network depth increasing, the accuracy becomes saturated and degrades rapidly → degradation problem

# Skip connections

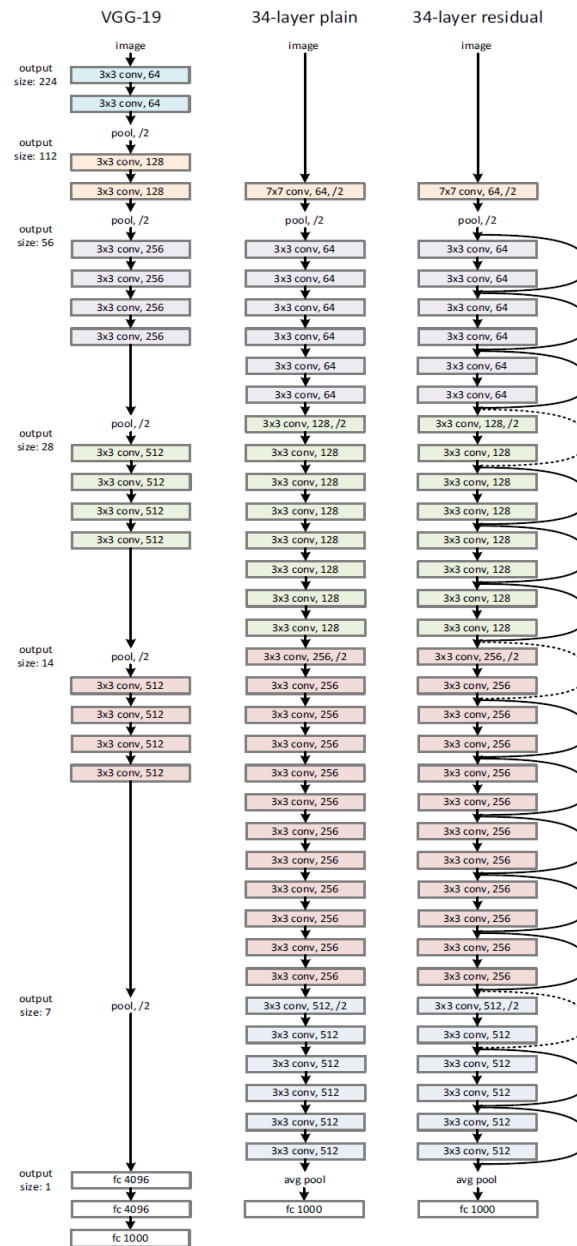


Deep residual networks for image recognition, He et al., 2015.

# ResNet (residual network)



# ResNet (residual network)



Deep residual networks for image recognition, He et al., 2015.

# ResNet (residual network)

| layer name | output size      | 18-layer                                                                    | 34-layer                                                                    | 50-layer                                                                                        | 101-layer                                                                                        | 152-layer                                                                                        |
|------------|------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| conv1      | $112 \times 112$ | $7 \times 7, 64, \text{stride } 2$                                          |                                                                             |                                                                                                 |                                                                                                  |                                                                                                  |
| conv2_x    | $56 \times 56$   | $3 \times 3 \text{ max pool, stride } 2$                                    |                                                                             |                                                                                                 |                                                                                                  |                                                                                                  |
|            |                  | $\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$   | $\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$   | $\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$    | $\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$     | $\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$     |
| conv3_x    | $28 \times 28$   | $\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$ | $\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$ | $\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$  | $\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$   | $\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$   |
| conv4_x    | $14 \times 14$   | $\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$ | $\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$ | $\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$ | $\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$ | $\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$ |
| conv5_x    | $7 \times 7$     | $\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$ | $\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$ | $\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$ | $\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$  | $\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$  |
|            | $1 \times 1$     | average pool, 1000-d fc, softmax                                            |                                                                             |                                                                                                 |                                                                                                  |                                                                                                  |
| FLOPs      |                  | $1.8 \times 10^9$                                                           | $3.6 \times 10^9$                                                           | $3.8 \times 10^9$                                                                               | $7.6 \times 10^9$                                                                                | $11.3 \times 10^9$                                                                               |

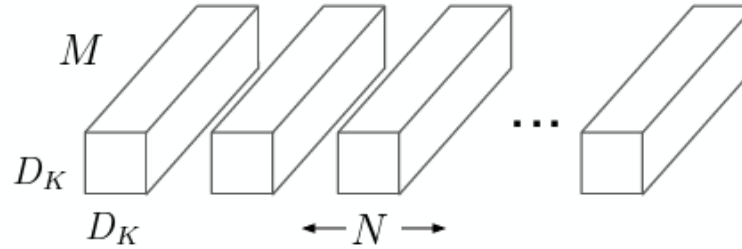
# MobileNet

| Type / Stride | Filter Shape                       | Input Size                           |
|---------------|------------------------------------|--------------------------------------|
| Conv / s2     | $3 \times 3 \times 3 \times 32$    | $224 \times 224 \times 3$            |
| Conv dw / s1  | $3 \times 3 \times 32$ dw          | $112 \times 112 \times 32$           |
| Conv / s1     | $1 \times 1 \times 32 \times 64$   | $112 \times 112 \times 32$           |
| Conv dw / s2  | $3 \times 3 \times 64$ dw          | $112 \times 112 \times 64$           |
| Conv / s1     | $1 \times 1 \times 64 \times 128$  | $56 \times 56 \times 64$             |
| Conv dw / s1  | $3 \times 3 \times 128$ dw         | $56 \times 56 \times 128$            |
| Conv / s1     | $1 \times 1 \times 128 \times 128$ | $56 \times 56 \times 128$            |
| Conv dw / s2  | $3 \times 3 \times 128$ dw         | $56 \times 56 \times 128$            |
| Conv / s1     | $1 \times 1 \times 128 \times 256$ | $28 \times 28 \times 128$            |
| Conv dw / s1  | $3 \times 3 \times 256$ dw         | $28 \times 28 \times 256$            |
| Conv / s1     | $1 \times 1 \times 256 \times 256$ | $28 \times 28 \times 256$            |
| Conv dw / s2  | $3 \times 3 \times 256$ dw         | $28 \times 28 \times 256$            |
| Conv / s1     | $1 \times 1 \times 256 \times 512$ | $14 \times 14 \times 256$            |
| 5×            | Conv dw / s1                       | $3 \times 3 \times 512$ dw           |
|               | Conv / s1                          | $1 \times 1 \times 512 \times 512$   |
|               | Conv dw / s2                       | $3 \times 3 \times 512$ dw           |
|               | Conv / s1                          | $1 \times 1 \times 512 \times 1024$  |
|               | Conv dw / s2                       | $3 \times 3 \times 1024$ dw          |
|               | Conv / s1                          | $1 \times 1 \times 1024 \times 1024$ |
|               | Avg Pool / s1                      | Pool $7 \times 7$                    |
|               | FC / s1                            | $1024 \times 1000$                   |
|               | Softmax / s1                       | Classifier                           |

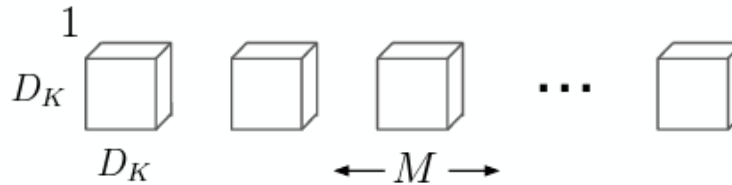
# MobileNet

Depth wise separable convolution:

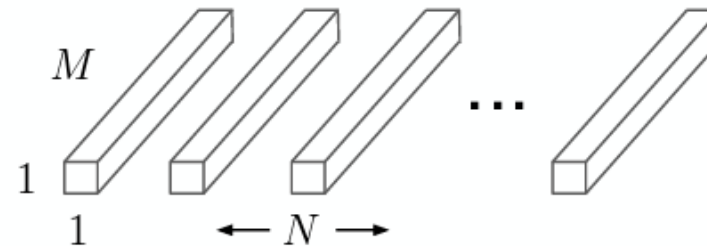
- depth wise convolutions
- pointwise convolutions



(a) Standard Convolution Filters

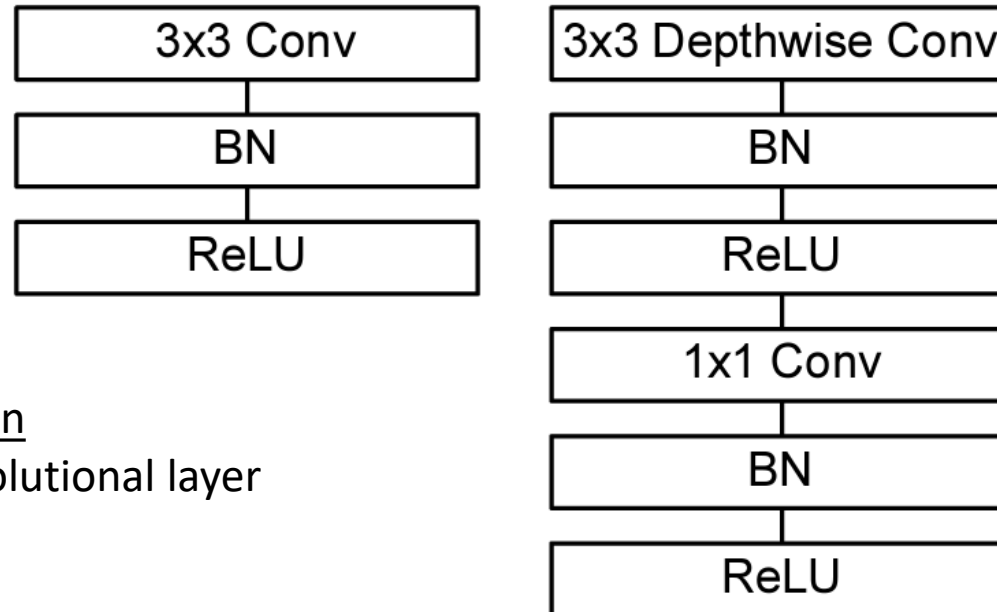


(b) Depthwise Convolutional Filters



(c)  $1 \times 1$  Convolutional Filters called Pointwise Convolution in the con-

# Mobilenet



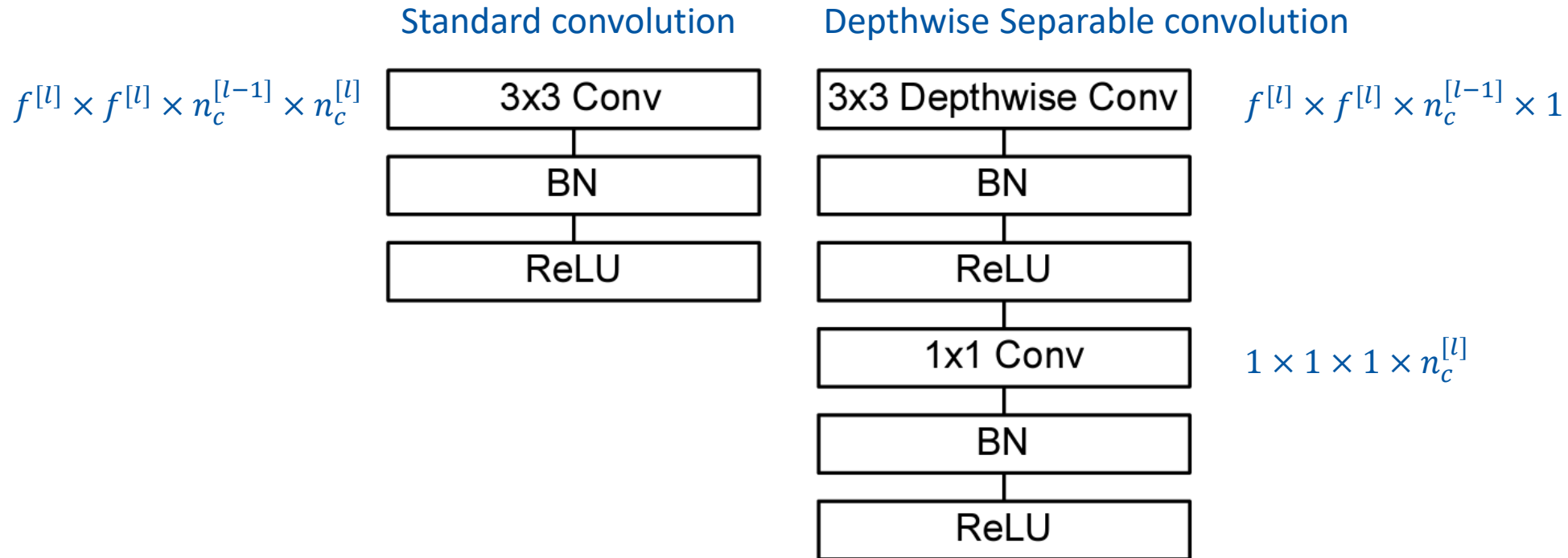
- Standard convolution

- Standard convolutional layer
- Batchnorm
- ReLU

- Depthwise Separable convolution

- Depthwise layer
- Batchnorm
- ReLU
- Pointwise layer
- Batchnorm
- ReLU

# Mobilenet



# Data Augmentation

Data augmentation is a technique of artificially increasing the training set by creating modified copies of a dataset using existing data.

It prevents overfitting, improves accuracy, and creates diversity in training data, which is crucial when datasets are small or imbalanced. By simulating real-world variations, it makes models more robust and reliable—without the need for expensive data collection.

augmentation techniques are not limited to images. You can augment audio, video, text, and other types of data too.

# Image Augmentation

**Geometric transformations:** randomly flip, crop, rotate, stretch, and zoom images.

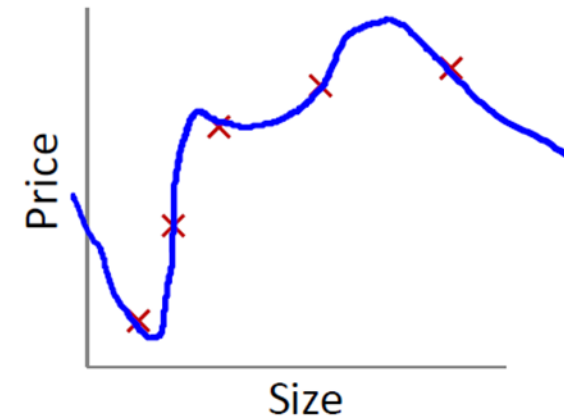
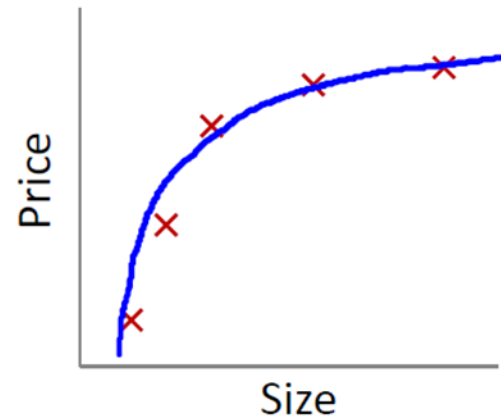
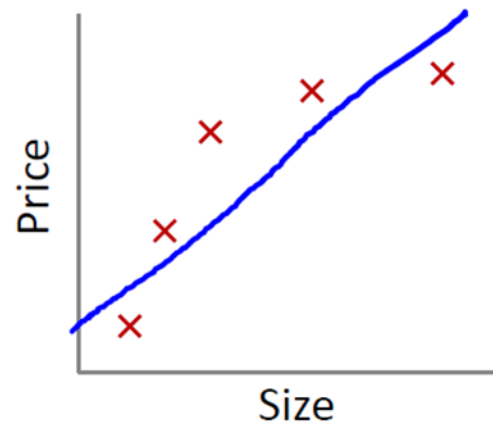
**Color space transformations:** randomly change RGB color channels, contrast, and brightness.

**Kernel filters:** randomly change the sharpness or blurring of the image.

**Random erasing:** delete some part of the initial image.

**Mixing images:** blending and mixing multiple images.

# Overfitting & Underfitting



## Overfitting:

a model performs well on the training data, but it does not generalize well.

## Underfitting:

it is the opposite of overfitting.

it occurs when a model is too simple to learn the underlying structure of the data.

# Split Train & Test data set

- A better option is to split your data into two sets: the training set and the test set.
- Train your model using the training set, and you test it using the test set.
- The error rate on new cases is called the **generalization error**.
- By evaluating your model on the test set, you get an estimate of this error.
- If the training error is low but the generalization error is high, it means that your model is **overfitting** the training data.

# Avoiding Overfitting through Regularization

- Sum of squares

**Ridge :  $L_2$  norm**

$$W_0^2 + W_1^2 + W_2^2 + \dots = \sum_{j=1}^n W_j^2 = \|W\|_2^2$$

Cost function  $+\lambda\|W\|_2^2$

- Sum of absolute value

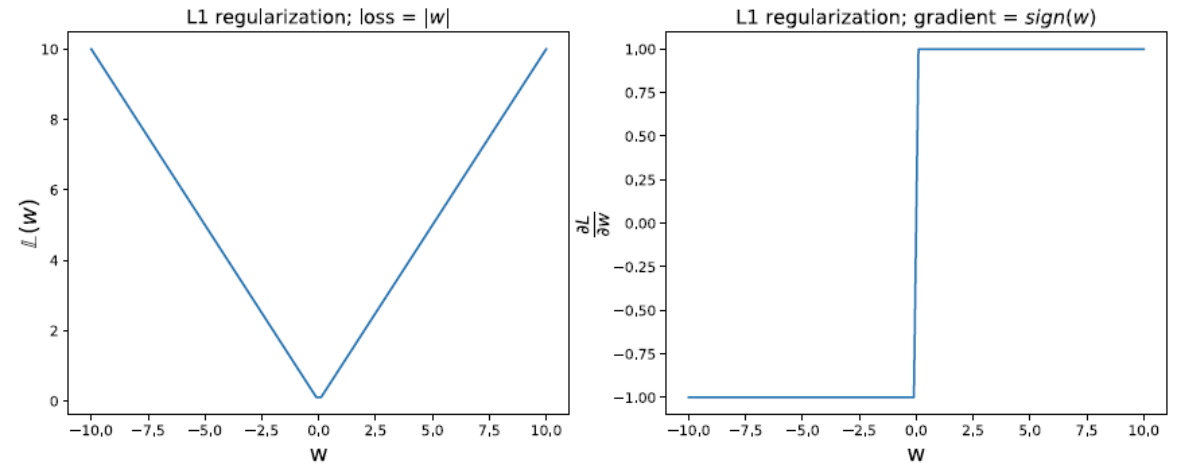
**Lasso:  $L_1$  norm**

$$|w_0| + |w_1| + |w_2| + \dots = \sum_{j=1}^n |w_j| = \|W\|_1$$

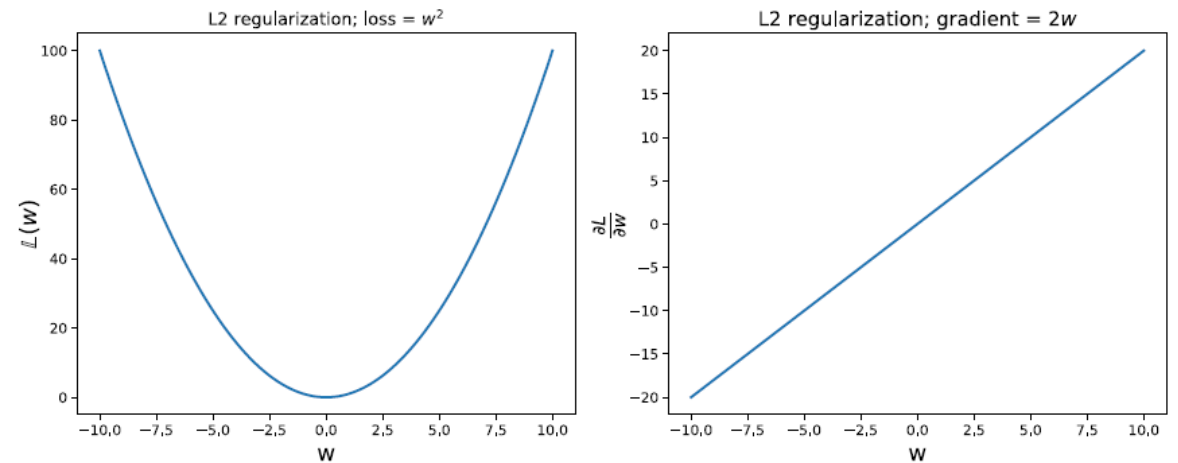
Cost function  $+\lambda\|W\|_1$

# L1. vs L2 regularization

- L1 regularization tends to create sparse models where many of the weights are 0.
- L2 regularization tends to create models with low (but nonzero) weights.



(a) L1 regularization



(b) L2 regularization

# Elastic net regularization

$$\textit{Cost function} + \lambda_1 |W| + \lambda_2 W^2$$

# Early Stopping

Stop training as the validation error reaches a minimum.

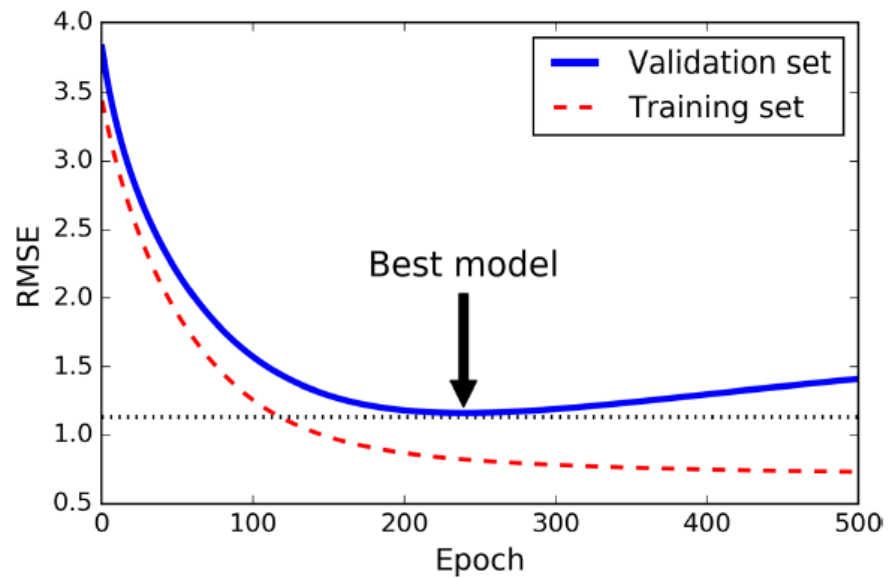


Image: Hands-On ML, Aurelien Geron

# Max norm constraints

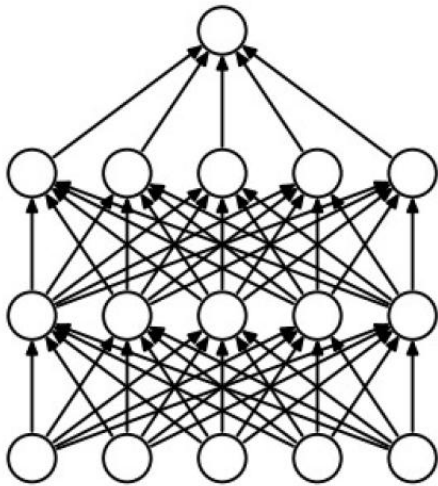
Enforce an absolute upper bound on the magnitude of the weight vector for every neuron.

$$\|W\|_2 < c$$

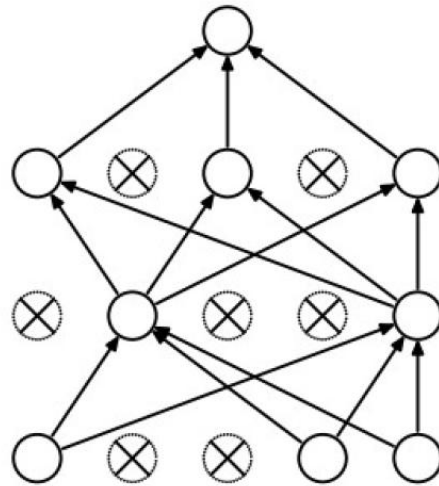
# Drop out

$$p \cdot 0 + (1 - p) \cdot x$$

Keep a neuron active with some probability (1-p) and set to zero others



(a) Standard Neural Net



(b) After applying dropout.

# Drop out

$$p \cdot 0 + (1 - p) \cdot x$$

Keep a neuron active with some probability (1-p) and set to zero others

